pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

JSON and the FRED API

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: a dated series the desk can trust

The Desk Lead's Note

Yesterday FRED's answer came back as text, its yields in quotes. Make it a dated series the desk can trust. Show me what is in the answer, and what FRED writes on a weekday with no yield. Prove a missing day never becomes a zero: show me what a zero would do. Put the request, the reading and the download in "marketdata.py", tested without the network, with errors that never show a key. Pull the 10-year and the 2-year from 2015 and tie them to the Treasury file, inside FRED's limit. Then read exactly what you are about to commit, and scan it for anything shaped like a key.

Every cell runs without a key; the saved outputs are that run, and every demo uses the course's fake key.

Start the Day

```
work = start_day(2)
Work folder pfi_course3_02_k4w7q2xr: 4 data files, Course 2's bondmath.py, marketdata.py as notebook 01 left it, 1 commit
```

The Module So Far

```
....   [100%]
4 passed in 0.50s
```

"start_day(2)" copied in the Treasury file, two answers in FRED's format, and notebook 01's committed files.

FRED's numbers arrive as text, and one of them is a dot: today it becomes "NaN", never 0.

2.1 JSON, read by Python

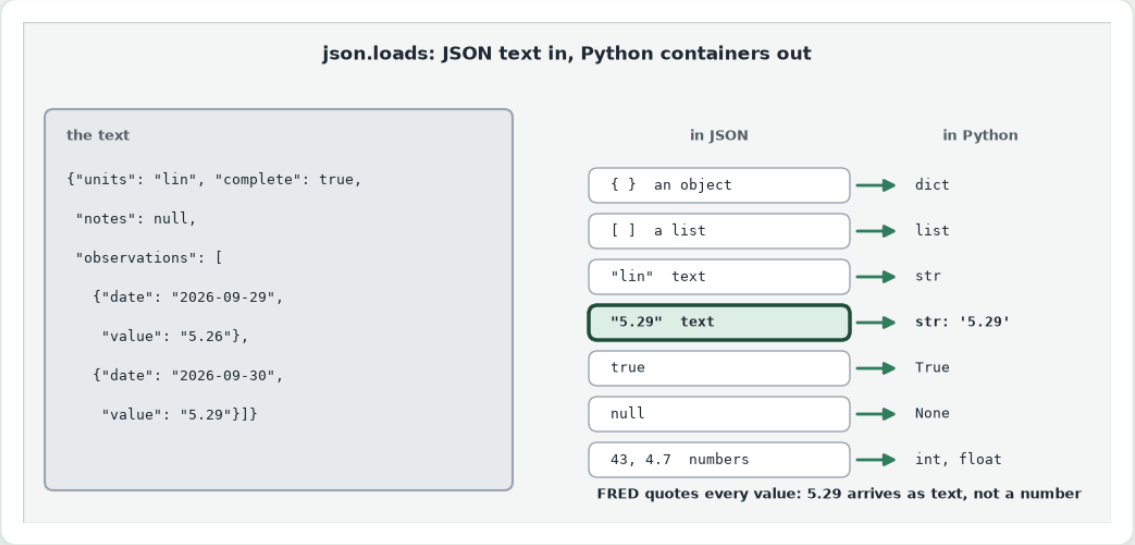
Notebook 01 ended on FRED's "5.29", in quotes: text, not yet a number. That answer is JSON, the text format web APIs answer in, and the one an AI assistant replies in when you ask it for structured data. JSON has two containers: an object "{ }" of "name": value pairs, and a list "[ ]" of values in order. Inside them sit text in quotes, numbers, "true", "false", and "null".

Two Days of the 10-Year, as JSON

```
text = '''{"units": "lin", "complete": true, "notes": null,
 "observations": [{"date": "2026-09-29", "value": "5.26"},
                  {"date": "2026-09-30", "value": "5.29"}]}'''
reply = json.loads(text)
print(type(reply), type(reply['observations']))
<class 'dict'> <class 'list'>
```

What Each Value Became

```
for name in ['units', 'complete', 'notes']:
    print(f'{name}: {reply[name]!r}, {type(reply[name]).__name__}')
latest = reply['observations'][-1]['value']
print(f'value: {latest!r}, {type(latest).__name__}')
units: 'lin', str
complete: True, bool
notes: None, NoneType
value: '5.29', str
```



An object became a "dict" and a list a "list". "5.29" stayed text: FRED quotes every value.

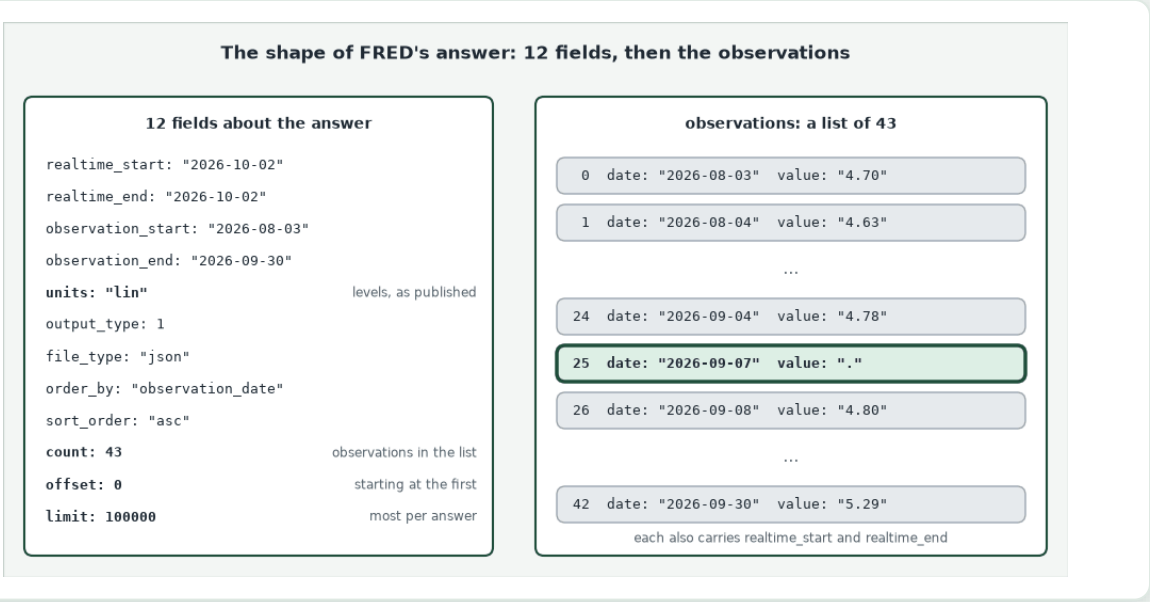
JSON is two containers and a few plain values; "json.loads" makes them a "dict" and a "list", and FRED's 5.29 arrives as the text '5.29'.

2.2 FRED's answer, field by field

Now the real answer. The course built this sample from the Treasury file in FRED's exact layout; nothing in it was downloaded from FRED.

Read the Sample

```
payload = json.loads(Path('course3_fred_format_dgs10.json').read_text(encoding='utf-8'))
print(list(payload))
print('count:', payload['count'])
['realtime_start', 'realtime_end', 'observation_start', 'observation_end', 'units', 'output_type', 'file_type', 'order_by', 'sort_order', 'count', 'offset', 'limit', 'observations']
count: 43
```



Twelve fields describe the answer; the 43 observations sit in a list of dicts. "count", "offset" and "limit": 43 observations, starting at the first, at most 100,000 to an answer.

The Observations as a Table

```
obs = pd.DataFrame(payload['observations'])
print(obs.dtypes)
print(len(obs), 'rows; count says', payload['count'])
realtime_start   str
realtime_end   str
date   str
value   str
dtype: object
43 rows; count says 43
```

Every column is text, "value" included.

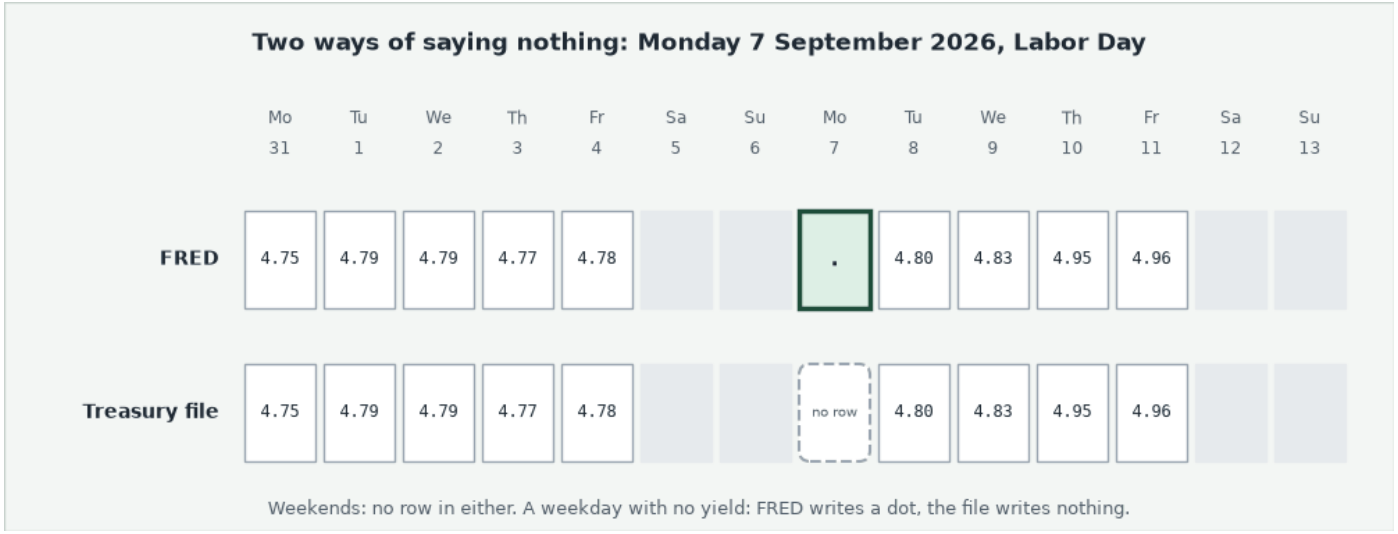
The days around Labor Day

The Days Around Labor Day

```
obs.loc[23:27, ['date', 'value']]
   date value
23  2026-09-03  4.77
24  2026-09-04  4.78
25  2026-09-07   .
26  2026-09-08  4.80
27  2026-09-09  4.83
```

Monday 7 September 2026 was Labor Day: no Treasury yield.

FRED writes a row whose value is a dot; the Treasury file writes no row.



Excel side by side: count the rows, count the dot

Excel side by side. Power Query reads the file (Data, Get Data, From Other Sources, Blank Query, Advanced Editor), every value as text. For the counts, B2:B44 of a sheet hold the 43 values: 42 numbers and the text ".".

Job	Python	Excel	Result
Read the answer	`pd.DataFrame(payload['observations'])`	`Json.Document(File.Contents(...)),`	43 rows, value as text, "." on 2026-09-07
		`then Table.FromRecords(Source[observations])`	
Count the rows	`len(obs)`	`=COUNTA(B2:B44)`	43
Count the numbers	`len(obs) - (obs['value'] == '.').sum()`	`=COUNT(B2:B44)`	42
Count the dots	`(obs['value'] == '.').sum()`	`=COUNTIF(B2:B44,".")`	1

FRED writes a row with a dot on a weekday with no yield, where the Treasury file writes no row at all: count rows and you count the dot.

Before the Web

FRED's own help pages say that "Before the popularization of the World Wide Web, the data were provided in list form on a dial-in, electronic bulletin board system." The same numbers now come back as JSON from a URL.

Source: FRED Help, "What is FRED?", https://fred.stlouisfed.org/help/about/about-fred/what-is-fred/ (read 2026-11-08).

2.3 The dot is a missing day

Can Python read the dot as a number?

Read the Dot as a Number

```
float('.')
ValueError: could not convert string to float: '.'
```

"pd.to_numeric" on the column stops at the same dot.

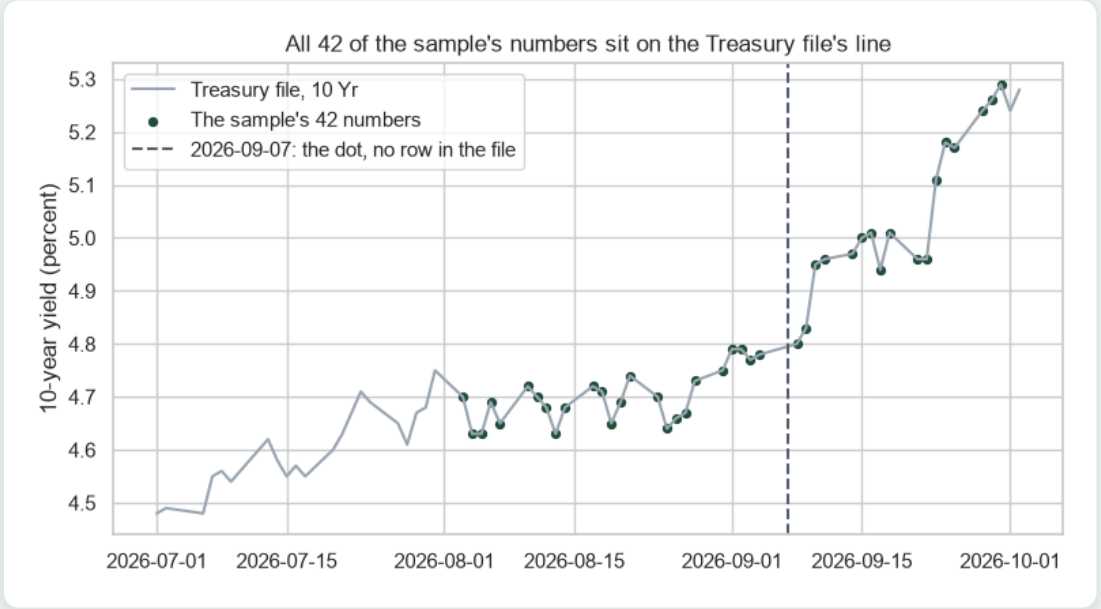
Mark It Missing

```
kept = pd.to_numeric(obs['value'], errors='coerce')
kept.index = pd.to_datetime(obs['date'])
print(kept.dtype, '|', kept.isna().sum(), 'NaN, on', kept.index[kept.isna()][0].date())
float64 | 1 NaN, on 2026-09-07
```

Drop It, Then Tie It to the Treasury File

```
sample_10y = kept.dropna()
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
gap = (sample_10y - treasury.loc[sample_10y.index, '10 Yr']).abs().max()
print(len(kept), 'rows,', len(sample_10y), 'after dropna; largest gap', gap, 'percent')
print('2026-09-07 in the Treasury file:', pd.Timestamp('2026-09-07') in treasury.index)
43 rows, 42 after dropna; largest gap 0.0 percent
2026-09-07 in the Treasury file: False
```

The Sample on the Snapshot



Excel side by side: the dot, missing

Excel side by side. B2 holds the text ".".

Job	Python	Result
Read the dot as a number	`float('.')`	#VALUE!
Mark it missing	`pd.to_numeric(..., errors='coerce')`	#N/A
Drop it	`kept.dropna()`	42 rows, every value a number

```
Read the dot as a number
=VALUE(".")
Mark it missing
=IFERROR(VALUE(B2),NA())
Drop it
Power Query Table.SelectRows(Rows, each [value] <> "."), before typing the column
```

42 numbers equal to the Treasury file, and the 43rd row is a day the market had no yield: the dot is a missing day, nothing else.

2.4 What a zero would do

The easy fix is a 0: the column stays numeric and nothing stops.

And the Average

```
print(f'Dot dropped: {sample_10y.mean():.4f} percent')
print(f'Dot as 0:    {as_zero.mean():.4f} percent')
print(f'Gap: {(sample_10y.mean() - as_zero.mean()) * 100:.1f} bp')
Dot dropped: 4.8367 percent
Dot as 0:   4.7242 percent
Gap: 11.2 bp
```

Excel side by side. Power Query loads the answer with its column typed; D is "value", so D26 is Friday, D27 Monday, D28 Tuesday.

```
Type the column
Table.TransformColumnTypes(Rows, {{"date", type date}, {"value", type number}}, "en-US")
From Friday
=(D28-D26)*100
A line chart
a line chart of the values
The average
=AVERAGE(B2:B44)
The average, dot as 0
=SUM(B2:B44)/ROWS(B2:B44)
```

Job	Python	Result
Type the column	`pd.to_numeric(..., errors='coerce')`	43 rows; the dot loads as an empty cell
From Friday	`sample_10y.diff() * 100`	1.9999999999999574
A line chart	`the two panels`	the text "." plotted at 0; an empty cell left as a gap, by default
The average	`sample_10y.mean()`	4.836666666666667: the text skipped
The average, dot as 0	`as_zero.mean()`	4.724186046511628

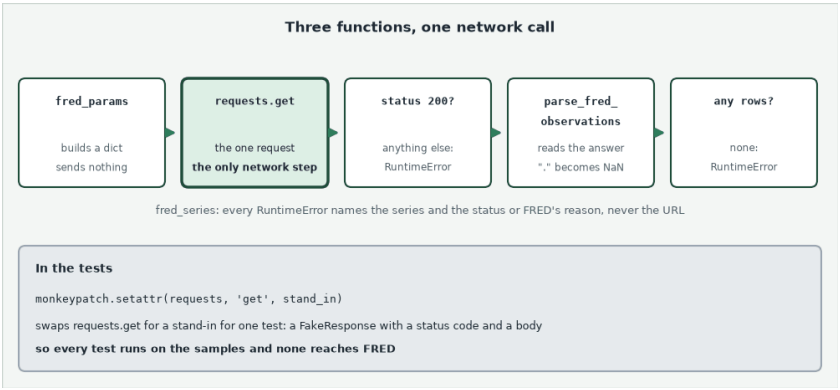
2.5 fred_params: the request, built and never sent

Now into the module, tested without the network.

```
%%writefile -a marketdata.py
import pandas as pd  # dated series, from day 2
import requests  # HTTP, from day 2
FRED_URL = "https://api.stlouisfed.org/fred/series/observations"
def fred_params(series_id: str, api_key: str, start: str | None = None, end: str | None = None) -> dict:
    """The query parameters for one request to FRED's series/observations endpoint.
    Units: start and end are dates as text, YYYY-MM-DD. Convention: JSON is asked for (file_type json);
    a start or end left as None is not sent, so FRED returns the series from its first or to its last date.
    Pure: builds a dict and sends nothing. The dict holds the key, so it is never printed.
    """
    params = {"series_id": series_id, "api_key": api_key, "file_type": "json"}
    if start is not None:
        params["observation_start"] = start
    if end is not None:
        params["observation_end"] = end
    return params
```

Read the docstring first. "str | None" marks an argument that may be "None", and "= None" makes it optional. The imports sit with the day's code; in your own file you may move them to the top.

"requests" joins these pairs after the "?", as notebook 01's first picture drew.



"fred_params" builds the request and "parse_fred_observations" reads the answer; neither touches the network. "fred_series", in 2.6, joins them around the one request.

The Sample's Parameters

```
importlib.reload(md)
params = md.fred_params('DGS10', FAKE_KEY, start='2026-08-03', end='2026-09-30')
print(list(params))
print('The key is the fake key:', params['api_key'] == FAKE_KEY)
['series_id', 'api_key', 'file_type', 'observation_start', 'observation_end']
The key is the fake key: True
```

parse_fred_observations, and three tests

Append parse_fred_observations

"pd.to_datetime(format=)" reads every date in one pattern. float("nan") makes a "NaN"; "pd.Series(values, index=, name=, dtype=)" names the series and fixes its type; .index.name = "date" names the index.

```
%%writefile -a marketdata.py
def parse_fred_observations(payload: dict, name: str) -> pd.Series:
    """Turn a FRED series/observations JSON answer into a float series.
    Units: as FRED publishes the series (percent for the Treasury and spread series).
    Convention: one value per observation, indexed by its date (a DatetimeIndex named date). FRED's
    missing marker "." becomes NaN and is kept: dropping it is a separate, visible step.
    Raises ValueError if the answer has no "observations" list.
    """
    if "observations" not in payload:
        raise ValueError(f"the answer for {name} has no 'observations'; keys are {sorted(payload)}")
    observations = payload["observations"]
    dates = pd.to_datetime([obs["date"] for obs in observations], format="%Y-%m-%d")
    values = [float("nan") if obs["value"] == "." else float(obs["value"]) for obs in observations]
    series = pd.Series(values, index=dates, name=name, dtype="float64")
    series.index.name = "date"
    return series
```

Parse the Sample

```
importlib.reload(md)
dgs10 = md.parse_fred_observations(payload, 'DGS10')
print(len(dgs10), 'rows,', dgs10.isna().sum(), 'NaN,', dgs10.dtype, '| index:', dgs10.index.name)
print('Same numbers as 2.3 by hand:', (dgs10.dropna() == sample_10y).all())
43 rows, 1 NaN, float64 | index: date
Same numbers as 2.3 by hand: True
```

Three Tests

"read_sample" reads a sample from the test file's own folder, "HERE", as in Course 2. "FakeResponse" is a class, as in Course 1's Classes notebook: a stand-in for FRED's answer, with a status code, a ".json()" method, and a URL holding the fake key. "math.isnan" asks whether a number is "NaN", which never equals itself, so "==" cannot.

```
def test_params_hold_series_and_json():
    params = md.fred_params("DGS10", FAKE_KEY, start="2015-01-01")
    assert params == {"series_id": "DGS10", "api_key": FAKE_KEY, "file_type": "json",
                      "observation_start": "2015-01-01"}
    assert "observation_end" not in params
def test_parse_sample_counts_and_nan():
    series = md.parse_fred_observations(read_sample("DGS10"), "DGS10")
    assert len(series) == 43
    assert series.isna().sum() == 1
    assert math.isnan(series.loc["2026-09-07"])
    assert series.loc["2026-09-30"] == 5.29
    assert series.dtype == "float64"
    assert series.index.name == "date" and series.name == "DGS10"
def test_parse_raises_without_observations():
    with pytest.raises(ValueError):
        md.parse_fred_observations({"error_code": 400, "error_message": "Bad Request."}, "DGS10")
```

```
.......   [100%]
7 passed in 0.50s
```

Two functions, neither touching the network: one builds the request, one reads the answer with the dot kept as "NaN", and "7 passed" proves both.

2.6 fred_series: errors that hide the key

```
%%writefile -a marketdata.py
def fred_series(series_id: str, api_key: str, start: str | None = None, end: str | None = None,
                timeout: float = 30) -> pd.Series:
    """Download one series from the FRED API: one GET with requests, then parse_fred_observations.
    Units: as FRED publishes the series. Convention: "." rows are kept as NaN, as parse_fred_observations.
    The only function in this module that uses the network.
    Errors never show the key or the URL (which holds the key):
    - it never calls response.raise_for_status(), whose message holds the full URL;
    - an answer other than HTTP 200 raises RuntimeError with the status and FRED's own error message;
    - a connection problem (requests.RequestException) raises RuntimeError naming only the exception class;
    - both are raised `from None`, so no chained traceback shows the original error and its URL.
    Raises RuntimeError if FRED answers with zero observations (FRED returns HTTP 200 with count 0 for a
    range the series does not cover).
    """
    try:
        response = requests.get(FRED_URL, params=fred_params(series_id, api_key, start, end), timeout=timeout)
    except requests.RequestException as error:
        raise RuntimeError(f"Request for {series_id} failed: {type(error).__name__}") from None
    status = response.status_code
    if status != 200:
        try:
            error_message = response.json().get("error_message", "no error message")
        except ValueError:
            error_message = "the answer was not JSON"
        error_message = error_message.replace(api_key, "***")
        raise RuntimeError(f"FRED returned HTTP {status} for {series_id}: {error_message}") from None
    series = parse_fred_observations(response.json(), series_id)
    if series.empty:
        raise RuntimeError(f"FRED returned no observations for {series_id} (start {start}, end {end})")
    return series
```

"fred_series" sends one request and hands the answer to the parser. FRED answers 200 with the observations, 400 with its reason for a bad request, 429 over its limit (notebook 01's table). Many tutorials and AI answers call "response.raise_for_status()", which raises for any status from 400 to 599: its message holds the whole URL.



Printed, logged, or pasted into a chat, the left one gives your key away.

Read the docstring first: its last lines are the promises. "except requests.RequestException" and "raise ... from None" work as in notebook 01's live cell: the original error, URL and all, is never printed.

Two tests that never reach FRED

```
def test_fred_series_error_hides_key(monkeypatch):
    def fake_get_400(url, params, timeout):
        return FakeResponse(400, {"error_code": 400,
                                  "error_message": "Bad Request.  The value for variable api_key is not registered."})
    monkeypatch.setattr(requests, "get", fake_get_400)
    with pytest.raises(RuntimeError) as caught:
        md.fred_series("DGS10", FAKE_KEY)
    printed = "".join(traceback.format_exception(caught.value))
    assert "HTTP 400" in str(caught.value) and "not registered" in str(caught.value)
    assert FAKE_KEY not in printed and "api_key=" not in printed
    def fake_get_down(url, params, timeout):
        raise requests.ConnectionError(f"Max retries exceeded with url: /fred/series/observations?series_id=DGS10"
                                       f"&api_key={FAKE_KEY}&file_type=json")
    monkeypatch.setattr(requests, "get", fake_get_down)
    with pytest.raises(RuntimeError) as caught:
        md.fred_series("DGS10", FAKE_KEY)
    printed = "".join(traceback.format_exception(caught.value))
    assert str(caught.value) == "Request for DGS10 failed: ConnectionError"
    assert caught.value.__cause__ is None and caught.value.__suppress_context__
    assert FAKE_KEY not in printed and "api_key=" not in printed
def test_fred_series_empty_raises(monkeypatch):
    empty = read_sample("DGS10") | {"count": 0, "observations": []}
    monkeypatch.setattr(requests, "get", lambda url, params, timeout: FakeResponse(200, empty))
    with pytest.raises(RuntimeError):
        md.fred_series("DGS10", FAKE_KEY, start="2010-01-01", end="2010-12-31")
    monkeypatch.setattr(requests, "get", lambda url, params, timeout: FakeResponse(200, read_sample("DGS10")))
    pd.testing.assert_series_equal(md.fred_series("DGS10", FAKE_KEY),
                                   md.parse_fred_observations(read_sample("DGS10"), "DGS10"))
```

monkeypatch.setattr(requests, "get", stand_in) swaps "requests.get" for a stand-in for one test. "traceback.format_exception" gives the text Python would print for an error. After "from None", "__cause__" is "None" and "__suppress_context__" is "True": Python keeps the original error but never prints it. "a | b" merges two dicts, the right one winning; "pd.testing.assert_series_equal" fails unless two series match in values, dates, dtype and name.

```
.........   [100%]
9 passed in 0.50s
```

```
importlib.reload(md)
print('fred_series is in the module:', 'fred_series' in dir(md))
fred_series is in the module: True
```

"fred_series" reports the status and FRED's own reason, never the URL, and a test that formats the whole error proves no key survives in it.

2.7 Live: the 10-year and the 2-year from 2015

This product uses the FRED® API but is not endorsed or certified by the Federal Reserve Bank of St. Louis.

FRED's API terms ask every application that uses the API to show that notice (https://fred.stlouisfed.org/docs/api/terms_of_use.html, read 2026-11-08). The key cell and live cells work as in notebook 01.

The Key Cell

```
try:
    fred_key = md.get_secret("FRED_API_KEY", env_file=ENV_FILE)
    print("FRED key: found. It is held in memory and is never printed.")
except RuntimeError as error:
    fred_key = None
    print(f"FRED key: not found. Live cells will be skipped. {error}")
FRED key: not found.
Live cells will be skipped.
Secret FRED_API_KEY not found.
Looked in: the environment variable FRED_API_KEY, Colab's Secrets panel, and the file .env given as env_file.
```

Pull and Tie

```
if fred_key is None:
    print(NO_KEY_MESSAGE)
else:
    for live_id, live_column in [('DGS10', '10 Yr'), ('DGS2', '2 Yr')]:
        live_series = md.fred_series(live_id, fred_key, start='2015-01-01')
        live_numbers = live_series.dropna()
        live_dots = live_series.index[live_series.isna()]
        live_common = live_numbers.index.intersection(treasury.index)
        live_gap = (live_numbers[live_common] - treasury.loc[live_common, live_column]).abs().max()
        assert len(live_common) > 0 and live_gap <= 1e-9, f'{live_id} and the file disagree'
        assert not live_dots.isin(treasury.index).any(), f'{live_id}: a dot on a file date'
        print(f'{live_id}: {len(live_series):,} rows, {len(live_dots)} dots,',
              f'last {live_series.index[-1].date()}; {len(live_common):,} dates in both,',
              f'largest gap {live_gap:.1e} percent')
Live cell skipped: no FRED key was found. Every other cell runs without one.
```

With your key, the first cell prints FRED's refusal, status and reason, and no key; the second prints one line per series: its rows since 2015, its dots, its last date, and the dates it shares with the file, at a largest gap of 0. The two sources publish the same numbers on different schedules, so the newest day can sit on one a day before the other; the check compares only dates both have.



With your key, both series pull through "fred_series" and equal the Treasury file on every date both have.

2.8 120 requests a minute, 2.9 read the staged change

2.8 120 Requests a Minute

FRED: "Up to 120 requests per minute are allowed before being served a 429 error code" (https://fred.stlouisfed.org/docs/api/fred/errors.html, read 2026-11-08). Each call of "fred_series" is one request, so a loop that pulls all 11 Treasury tenors sends 11 a run.

What Changed, Then Stage It

The tests read the sample, which "start_day" copies every day: it stays out of the record.

```
!git -C "{work}" status --short
!git -C "{work}" add marketdata.py test_marketdata.py
 M marketdata.py
 M test_marketdata.py
```

After three lines of context, every line starts with "+": today added code and changed none.

What the Commit Will Record, Line by Line

```
!git -C "{work}" diff --cached --stat
!git -C "{work}" diff --cached marketdata.py
 marketdata.py   | 84 ++++++++++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py | 94 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 178 insertions(+)
diff --git a/marketdata.py b/marketdata.py
index 9a24a8c..f15a4f2 100644
--- a/marketdata.py
+++ b/marketdata.py
@@ -56,3 +56,87 @@ def get_secret(name: str, env_file: str | Path) -> str:
    f"Secret {name} not found. Looked in: the environment variable {name}, "
    f"Colab's Secrets panel, and the file {env_path.name} given as env_file."
    )
+
+
+import pandas as pd  # dated series, from day 2
+import requests  # HTTP, from day 2
```

Scan for a key, then commit

"git grep" searches the files Git tracks; "--cached", as in "diff", reads the staged copies, exactly what the commit will hold. "-n" adds line numbers, and "-E" reads the pattern as in Course 1's Analyzing Text: "[a-z0-9]" one lowercase letter or digit, "{32}" thirty-two in a row, a key's shape.

```
# IPython reads { } in a ! line as Python, so the braces are doubled
!git -C "{work}" grep --cached -n -E "[a-z0-9]{{32}}"
test_marketdata.py:12:FAKE_KEY = "fakefakefakefakefakefakefakefake"
```

One hit, the course's fake key, where the tests keep it: expected. A real key found only staged: take it out of the file and stage again, or unstage with Course 2's "git restore --staged". A real key already in a commit: rotate it first, as notebook 01's drill showed.

Commit

```
subject = 'Notebook 02: fred_params, parse_fred_observations, and fred_series'
body = 'Pure functions build the request and read the answer; fred_series never shows the key.'
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
!git -C "{work}" log --oneline
f3002d2 Notebook 02: fred_params, parse_fred_observations, and fred_series
a28686d Notebook 01: get_secret, a .gitignore, and .env.example
```

The commit holds two files, read line by line and scanned: one 32-character run, the course's fake key, where the tests keep it.

Bring it home: into my_bondmath

Copy the work folder's two files into "my_bondmath", with the two samples the tests read, test them, scan, and commit. The copy replaces your two files, so anything of your own must be in the work folder first. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
$work = "$env:TEMP\pfi_course3_02_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
Copy-Item "$work\course3_fred_format_dgs10.json", "$work\course3_fred_format_dgs2.json" .
python -m pytest -q test_marketdata.py
git add marketdata.py test_marketdata.py
git diff --cached --stat
git grep --cached -n -E "[a-z0-9]{32}"
git commit -m "Notebook 02: fred_params, parse_fred_observations, and fred_series"
```

pytest shows "9 passed"; in PowerShell the braces are single, and "git grep" never reads your ignored ".env". Restart this notebook and run all cells: with your key, the live cells pull FRED. In Colab, download the two files from the work folder under "/tmp".

You Can Now

Read JSON with "json.loads", and say what each piece becomes in Python.

Find FRED's missing marker, the dot, and say why the Treasury file has no row there.

Keep a missing day as "NaN", drop it in a step you can see, and show what a 0 would do.

Build a request with "fred_params" and read an answer with "parse_fred_observations", tested without the network.

Download a series with "fred_series", whose errors never show the key, proved with "monkeypatch".

Tie a live pull to the Treasury file, and stay inside FRED's 120 requests a minute.

Read a staged change with "git diff --cached", and scan it for a key with "git grep".

Predict first: Python's text, and the dot as 0

Predict 1

A Python dictionary, printed, looks a lot like JSON. Hand that printed text to "json.loads":

A) it reads it

B) it stops with an error

C) it reads it with every value as text

```
json.loads(str(reply))
```

Predict 2

Read the dot as 0. Friday 4 September is 4.78 and Tuesday 8 September 4.80. What change does Tuesday show, in bp? Write down a number.

```
as_zero = kept.fillna(0.0)
changes = pd.DataFrame({'dot as 0': as_zero.diff() * 100, 'dot dropped': sample_10y.diff() * 100})
changes.loc['2026-09-03':'2026-09-08'].round(2)
```

Predict first: the NaN, the limit, and git diff

Predict 3

Keep the "NaN" and call ".diff()" straight away. Tuesday shows:

A) 2 bp: pandas looks back to Friday

B) "NaN"

C) 480 bp: "NaN" counts as 0

```
(kept.diff() * 100).loc['2026-09-04':'2026-09-08'].round(2)
```

Predict 4

You run that loop again and again. On which run within one minute does FRED first answer 429? Write down a number.

```
runs = list(range(1, 13))
requests_made = [11 * run for run in runs]
colours = ['#1E4D3A' if made > 120 else '#9AA5B1' for made in requests_made]
```

Predict 5

The two files are staged. What does plain "git diff --stat" print now?

A) the same two files

B) nothing

C) an error: there is no change

```
!git -C "{work}" diff --stat
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: parse_yields's test, written from its docstring, fails: AssertionError: a yield of 0 percent.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
What is in the answer	12 fields and 43 observations, every value text
A weekday with no yield	A dot, on 2026-09-07, a day the Treasury file has no row for
Three functions, tested	fred_params, parse_fred_observations, fred_series, 9 passed
Errors without a key	The status and FRED's reason, raised from None
The 10-year and 2-year live	Tied to the file on every common date
Read and scanned	git diff --cached, one expected hit

Be proud: every FRED number in this course now comes through functions you wrote and tested.

Next: Market Calendars and Cleaning (notebook 03) asks which days the market really had.